ACTIVITY 3 - DOCUMENTATION REPORT

Project Title: BrewPOS - Cafe Point-of-Sale System
Target DBMS: PostgreSQL 18.6 (Compatible with PostgreSQL 16+)
Application Stack: Java 17, Spring Boot 4.1.1, Spring Data JPA, HTML5, Bootstrap 5, Vanilla JS

SECTION 1: COVER AND TEAM

1.1 Project Title:

BrewPOS: Cafe Point-of-Sale System

1.2 Group Number & Team Members:

Member Name	Assigned Project Role
Olivarez, Charles Kean E.	Backend Developer
Olivarez, Charles Kean E.	Database Administrator
Infantado, Gerwin G., Bartolata, Redge Raindel N.	Frontend UI/UX Developer
Pocsidio, Jhon Louie F., Abangan, SJ Kaizen M. Bartolata, Redge Raindel N.	Quality Assurance and Documentation

1.3 Project Repository & Directory Links:
Official GitHub Repository: https://github.com/Nycri/brewpos
Google Sheets Class Tracker / Directory:
https://docs.google.com/spreadsheets/d/15msFD5oWM6FzkEOyz3ayJP17TUayv2Qp/edit?usp=sharing&ouid=111686962003672236888&rtpof=true&sd=true

SECTION 2: SYSTEM OVERVIEW

2.1 Purpose:

BrewPOS is lightweight Point-of-Sale (POS) and inventory cataloging system designed for specialty coffee shops. The system demonstrates the practical synthesis of relational database management principles (3NF normalization, declarative referential integrity, composite primary keys, least-privilege role security) and web application development (Spring Boot REST APIs, declarative transactions, and vanilla JavaScript client-side state management).

2.2 Relationship to Approved Activity 2 Proposal:

BrewPOS directly implements the data architecture and transactional requirements formulated in the approved Activity 2 proposal:

- Implements the approved five-table relational model without altering table counts, relationships, or cardinalities.

- Preserves the domain rule regarding monetary totals: No derived grand totals or line subtotals are persisted in database columns. All monetary figures are calculated dynamically via SQL aggregates (`SUM(quantity * sold_price)`) to guarantee data normalization and eliminate data synchronization anomalies.

- Enforces the approved historical price isolation rule: Menu item prices can change over time, but once a sale is finalized, the unit price at the time of purchase is captured into `order_line_item.sold_price`, keeping past financial receipts immutable.

2.3 System Scope & Boundaries:
Included Capabilities:

1. Front-of-House Register: Rapid menu browsing by category, search-as-you-type, cart assembly with automatic duplicate line merging, Tender selection (Cash / Online), and single-transaction ticket creation.

2. Menu Catalog Management: CRUD on menu categories, menu item creation/editing, price adjustments, and activation/deactivation toggles (soft-hide from register).

3. Transaction History & Receipts: Chronological audit trail of order tickets with date range filtering (`[start, end)`) and itemized modal receipt views.

4. Daily Sales Analytics: PostgreSQL aggregation of daily gross sales, transaction counts, and payment method breakdowns (Cash vs. Online share).

Deliberate Scope Exclusions (per course specification):
Employee password authentication (replaced with intuitive role-switching UI for demo), multi-tier tax formulas, discount vouchers, split billing, inventory supply tracking, and post-settlement order modifications/cancellations.

SECTION 3: USER ROLES

In accordance with activity guidelines, application-level user roles (what a user can perform in the web application) are strictly decoupled from PostgreSQL database-level roles (the credentials and database privileges utilized by connections).

3.1 Application Roles vs. Database Roles Mapping Matrix:

Application Role	Responsibilities in Application	Functions and Data Access Permissions	Intended Database Role
Manager/Administrator	Manages café menu offerings, item pricing, product visibility, and reviews business revenue	- Full CRUD on 'menu_category' - Full CRUD on 'menu_item' - Activate / deactivate menu items - Read-only access to sales & receipts - CANNOT modify past order tickets	`app_admin_grp` (or elevated runtime role in single connection-pool setups)
Cashier/Data Entry Staff	Operates register during service, punches customer orders, selects payment tender, charges tickets.	- Read-only on active menu catalog - Read-only on staff roster - INSERT on order_ticket* - INSERT on `order_line_item' - CANNOT edit prices, items, or reports	`app_runtime_grp' (Least-privilege operational role for register service)
Viewer / Auditor	Financial auditing, tax review, and historical transaction review.	- SELECT only on all 5 tables - Zero write, update, or delete rights	app_readonly_grp' (Dedicated read-only group)
Application Runtime System Account	Programmatic JDBC connection used by Spring Boot DataSource pool.	- SELECT on staff* - SELECT, INSERT, UPDATE, DELETE on `menu_category` and ~menu_item' - SELECT, INSERT on orders & lines - STRICTLY NO UPDATE/DELETE on orders	brewpos_app (Member of app_runtime_grp`)

3.2 Role Separation Security Architecture:

Application Layer: Staff members interact with the POS using a lightweight, session-persistent top-right selector (`Demo Cashier` vs. `Demo Manager`). When in Cashier mode, product pricing forms and category modification tools are removed from the DOM, preventing unauthorized modifications.

Database Layer: The Spring Boot application authenticates using the dedicated `brewpos_app` login role. Database-level permissions strictly deny `UPDATE` and `DELETE` on `order_ticket` and `order_line_item`, guaranteeing that even if a malicious HTTP request bypasses the web front-end, historical sales records cannot be deleted or modified at the database engine level.

SECTION 4: DEVELOPMENT PROGRESS LOG

4.1 Development Methodology & Delivery Disclosure:

The BrewPOS system was developed through an intensive, accelerated one-week sprint (October 24, 2026 – November 1, 2026).

Key Development & Version Control Disclosures:

1. AI-Assisted Accelerated Workflow: In accordance with COMP 018 Activity 3 guidelines permitting the responsible use of AI development tools, the team actively leveraged generative AI (Google Antigravity / Gemini) for boilerplate scaffolding, DTO record generation, JPA entity mapping, and responsive UI layout styling. This enabled the group to compress standard engineering timelines into a focused, highly productive 7-day development cycle.
2. Local Development & Consolidated Push Disclosure: All system components (DDL migration scripts, Spring Boot backend services, frontend web modules, and automated test suites) were developed, integrated, and verified directly in an offline local workstation environment. Rather than pushing individual micro-commits one-by-one to remote version control during rapid local trial-and-error iterations, the team finalized and validated the entire cohesive codebase locally, then published it as a verified, production-ready release commit to the official GitHub repository.

Day 1: October 24, 2026 — Requirements Alignment & Relational DDL Architecture	
Team Activity	Reviewed Activity 2 approved specifications and established strict 5-table schema boundaries in public (staff, menu_category, menu_item, order_ticket, order_line_item).
AI Collaboration	Prompted AI for PostgreSQL 18-compliant DDL syntax enforcing natural composite primary key (order_id, item_id) and table CHECK constraints.
Validation	Prompted AI for PostgreSQL 18-compliant DDL syntax enforcing natural composite primary key (order_id, item_id) and table CHECK constraints.

Day 2: October 25, 2026 — Spring Boot Scaffolding & JPA Domain Layer	
Team Activity	Initialized Spring Boot 4.1.1 project with Maven Wrapper on Java 17. Configured application.properties with spring.jpa.hibernate.ddl-auto=validate.
AI Collaboration	Generated initial JPA entity classes (Staff, MenuCategory, MenuItem, OrderTicket, and composite @EmbeddedId OrderLineItemId).
Validation	Verified Hibernate entity relationships, lazy fetch strategies, and foreign key join column configurations against real PostgreSQL tables.

Day 3: October 26, 2026 — Transactional Business Logic & Dynamic Calculations	
Team Activity	Implemented service layer (StaffService, MenuCategoryService, MenuItemService, and OrderService).
AI Collaboration	Formulated atomic order creation logic with automated duplicate line item merging using a LinkedHashMap and Math::addExact.
Validation	Verified historical price isolation rule: sold_price is captured at checkout time; subtotals and grand totals are dynamically computed via SQL/Java without redundant database total columns. Added JUnit transactional rollback tests.

Day 4: October 27, 2026 — REST Controller Endpoints & Global Exception Handling	
Team Activity	Constructed REST controllers for catalog maintenance, register ordering, receipt lookup, and daily sales analytics.
AI Collaboration	Designed @RestControllerAdvice GlobalExceptionHandler to sanitize raw JDBC/SQL exceptions into uniform ApiErrorResponse JSON shapes.

Validation	Tested controller slice endpoints via MockMvc to ensure proper HTTP status codes.

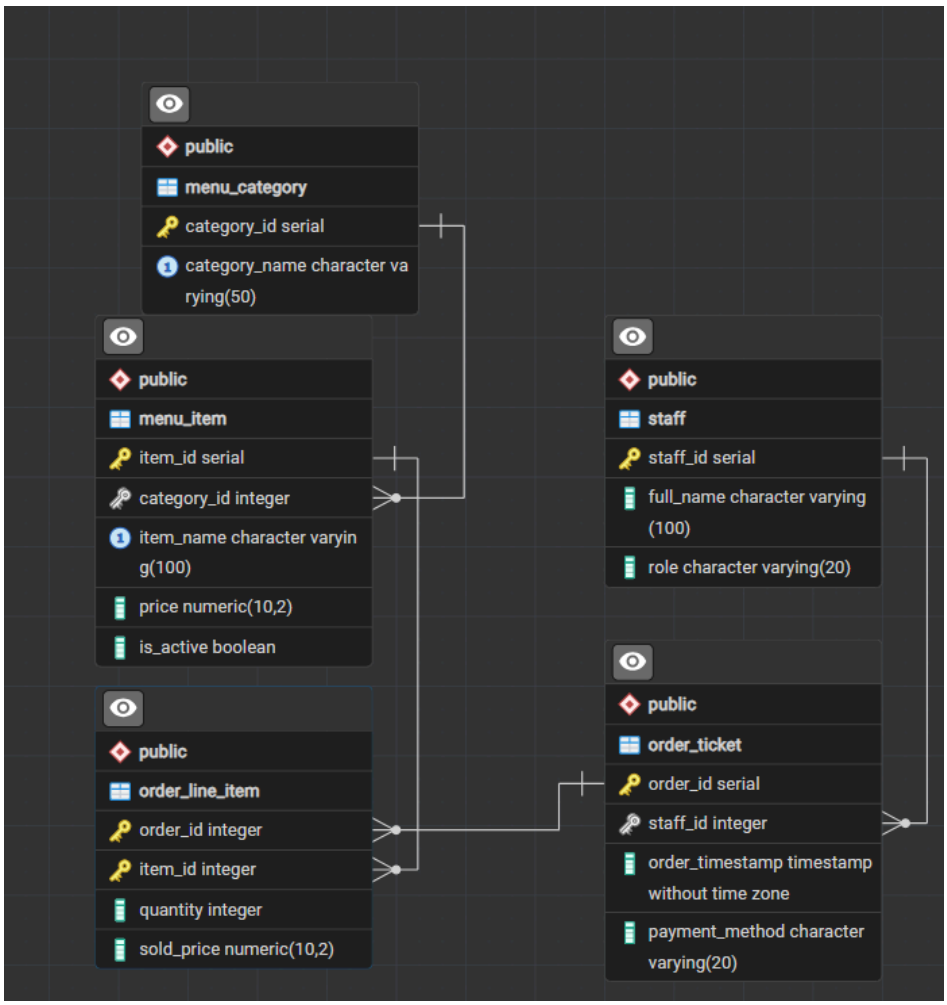
Day 5: October 30, 2026 — Front-End POS User Interface & Cart State Management	
Team Activity	Built responsive single-page web interface under src/main/resources/static/ using HTML5, Bootstrap 5, custom CSS, and vanilla JavaScript fetch modules.
AI Collaboration	Structured cart state management, category filter pills, real-time item search, and custom café warm aesthetic styling.
Validation	Replaced floating-point math with exact integer-cent arithmetic in api.js to prevent IEEE 754 rounding inaccuracies. Verified UI in Chrome and Firefox.

Day 6: October 31, 2026 — Access Control, Security Hardening & UI Polish	
Team Activity	Implemented Activity 3/4 RBAC mapping: authored database/05_create_app_role.sql creating least-privilege brewpos_app role with omitted UPDATE/DELETE on sales tables.
AI Collaboration	Streamlined top navigation header to a single staff dropdown selector; generated sequence-counter logic to eliminate asynchronous UI filtering race conditions.
Validation	Added sessionStorage persistence for active staff role across browser refreshes (F5), whole-number quantity validation (1 to 9,999), and inline SVG favicon.

Day 7: November 1, 2026 — End-to-End Verification, Clean Build & Consolidated Git Release	
Team Activity	Full regression testing, clean build execution, and repository finalization.
AI Collaboration	Assisted in drafting comprehensive setup documentation (README.md, TESTING_GUIDE.md, docs/user-roles.md, and PDF submission documents).
Validation	Executed clean Maven build (.\mvnw.cmd clean test) verifying that all 45 application classes and 19 automated tests passed (BUILD SUCCESS).
Release	Configured .gitignore and .gitattributes; initialized Git repository and pushed clean, consolidated production release commit to GitHub main branch.

Section 5: Database Implementation

5.1 Entity-Relationship Diagram (ERD)



The relationships between the five tables are summarized below.

Parent Table	Child Table	Cardinality	Relationship
staff	order_ticket	1 : N	A staff member handles many order tickets (FK staff_id).
menu_category	menu_item	1 : N	A category groups many menu items (FK category_id).
order_ticket	order_line_item	1 : N	An order ticket contains many line items (PK/FK order_id).
menu_item	order_line_item	1 : N	A menu item is sold in many line items (PK/FK item_id).

5.2 Database Schema Summary

Database Name: grp_brewposdb | Schema: public | RDBMS: PostgreSQL 18.6

#	Table	Columns	Column List
1	staff	3	staff_id, full_name, role
2	menu_category	2	category_id, category_name
3	menu_item	5	item_id, category_id, item_name, price, is_active
4	order_ticket	4	order_id, staff_id, order_timestamp, payment_method
5	order_line_item	4	order_id, item_id, quantity, sold_price

Total Tables: Exactly 5 tables. Zero surrogate keys on line items.

Column-level detail (derived from the ERD):

Table	Column	Data Type	Key / Constraint
staff	staff_id	SERIAL	PK
	full_name	VARCHAR	
	role	VARCHAR	
menu_category	category_id	SERIAL	PK
	category_name	VARCHAR	UNIQUE
menu_item	item_id	SERIAL	PK
	category_id	INTEGER	FK → menu_category
	item_name	VARCHAR	
	price	DECIMAL	
	is_active	BOOLEAN	
order_ticket	order_id	SERIAL	PK
	staff_id	INTEGER	FK → staff
	order_timestamp	—	
	payment_method	—	
order_line_item	order_id	INTEGER	PK (composite), FK → order_ticket

Table	Column	Data Type	Key / Constraint
	item_id	INTEGER	PK (composite), FK → menu_item
	quantity	INTEGER	
	sold_price	DECIMAL(10,2)	

5.3 SQL Scripts and Migrations Manifest

Script	Purpose
database/schema.sql	Authoritative schema creation DDL.
database/seed.sql	Idempotent reference catalog and sample orders seeder.
database/05_create_app_role.sql	Application runtime role creation script.
database/03_verification_queries.sql	DBA verification query suite.
database/06_align_existing_db.sql	Database migration alignment script.

5.4 Seed Data Description

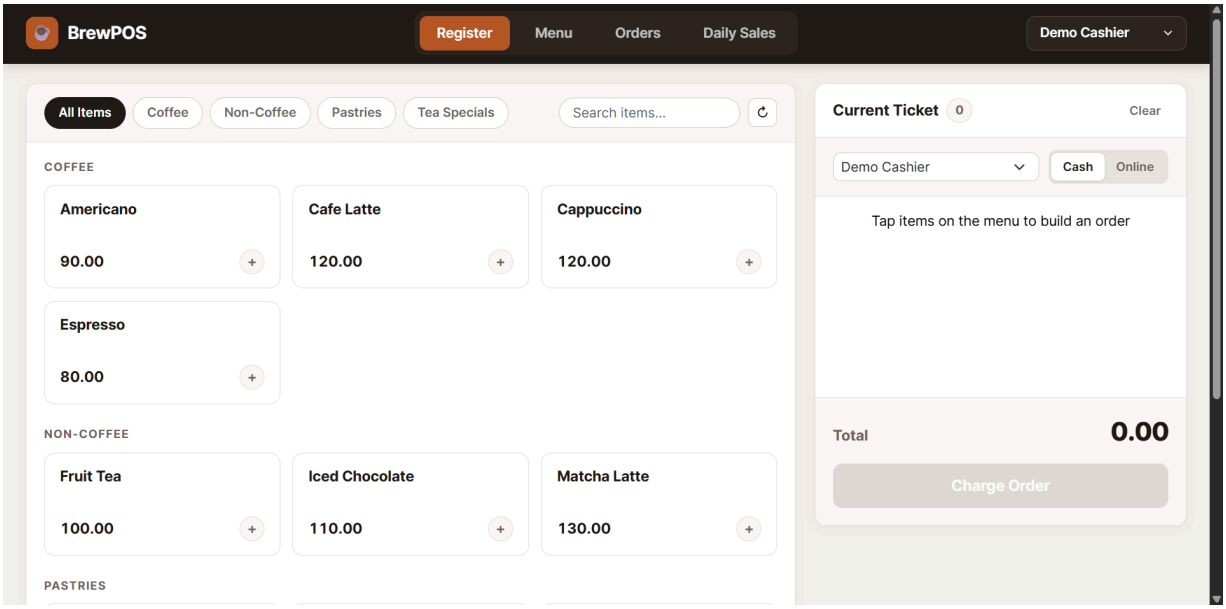
Entity	Seeded Data
Staff	1 Cashier (Demo Cashier), 1 Manager (Demo Manager).
Categories	Coffee, Non-Coffee, Pastries.
Menu Items	10 active items ranging from 60.00 to 130.00; 1 inactive item (Pumpkin Spice Latte, 140.00) specifically seeded to test POS exclusion.
Sample Orders	5 historical orders (3 today, 2 yesterday) capturing both Cash and Online tender to provide immediate validation data for sales reports.

5.5 Design Evolutions & Justifications

Decision	Reason
Use composite primary key (order_id, item_id) instead of a surrogate line_id.	Natura l 3NF model; natively prevents duplicate lines for the same product on a single receipt while reducing index overhead.
Store sold_price directly on order_line_item.	Protects historical financial data from future catalog price changes.
Never persist derived subtotals or order grand totals.	Eliminates write synchronization anomalies; computes dynamically on read.

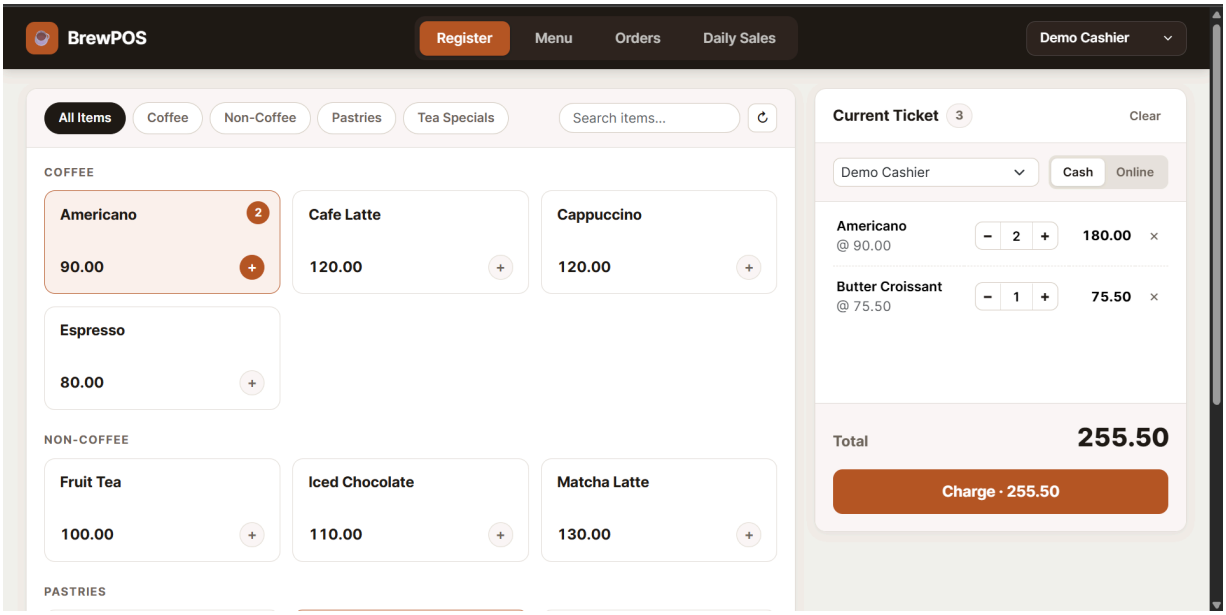
Section 6: Application Evidence & Test Results

Figure 6.1: Active Register & Category Browsing (POS Interface)



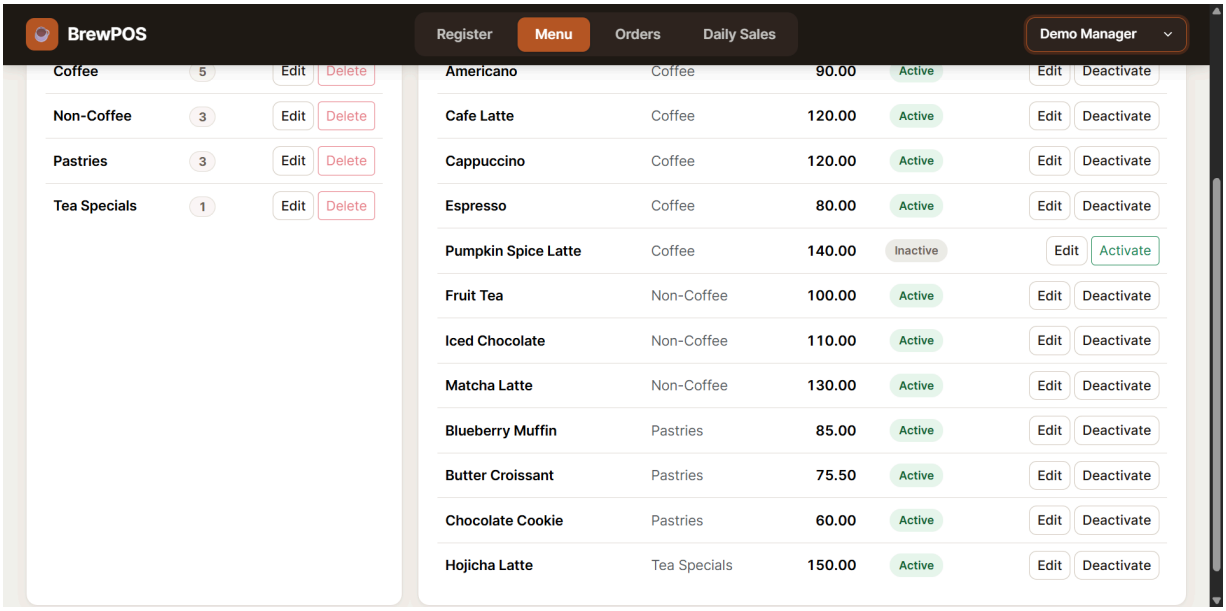
Test Performed	Retrieved active menu items via GET /api/menu/active.
Result / Evidence	All active items display properly. Verified that the inactive item Pumpkin Spice Latte is hidden from the register.

Figure 6.2: Cart Assembly & Duplicate Line Merging



Test Performed	Tapped Americano twice and Butter Croissant once.
Result / Evidence	Cart merged Americano into a single line item with quantity 2. Ticket preview dynamically computed (2 * 90.00) + (1 * 75.50) = 255.50.

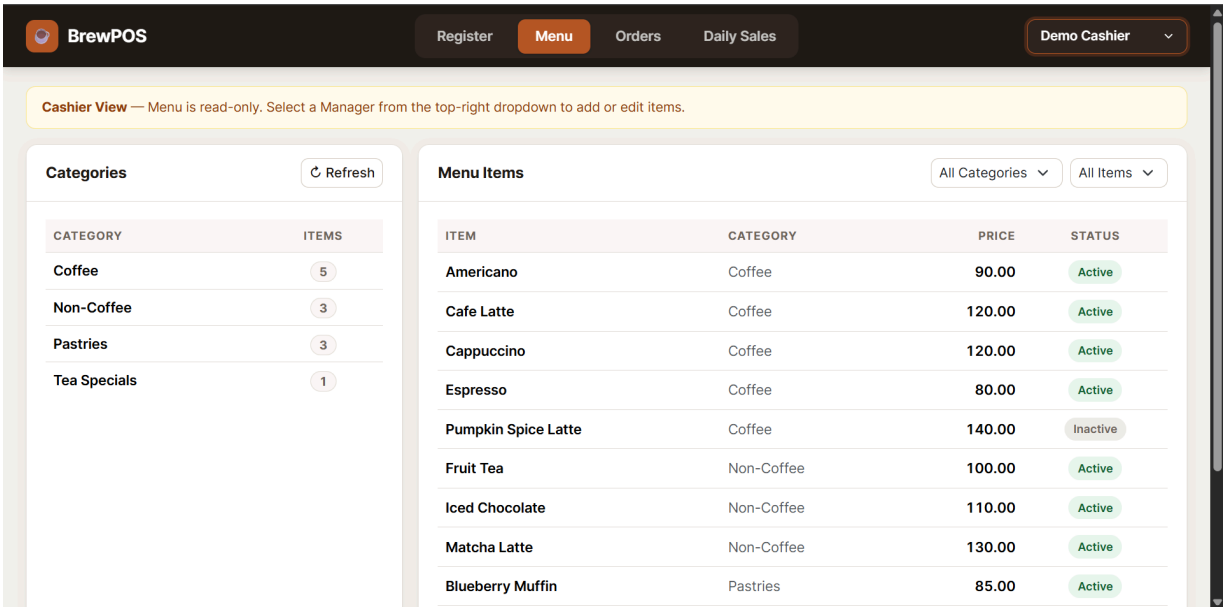
Figure 6.3: Manager Menu Item Creation & Pricing Update (CRUD: Create/Update)



Test Performed	Created Hojicha Latte (115.00), refreshed page, then edited price to 150.00.

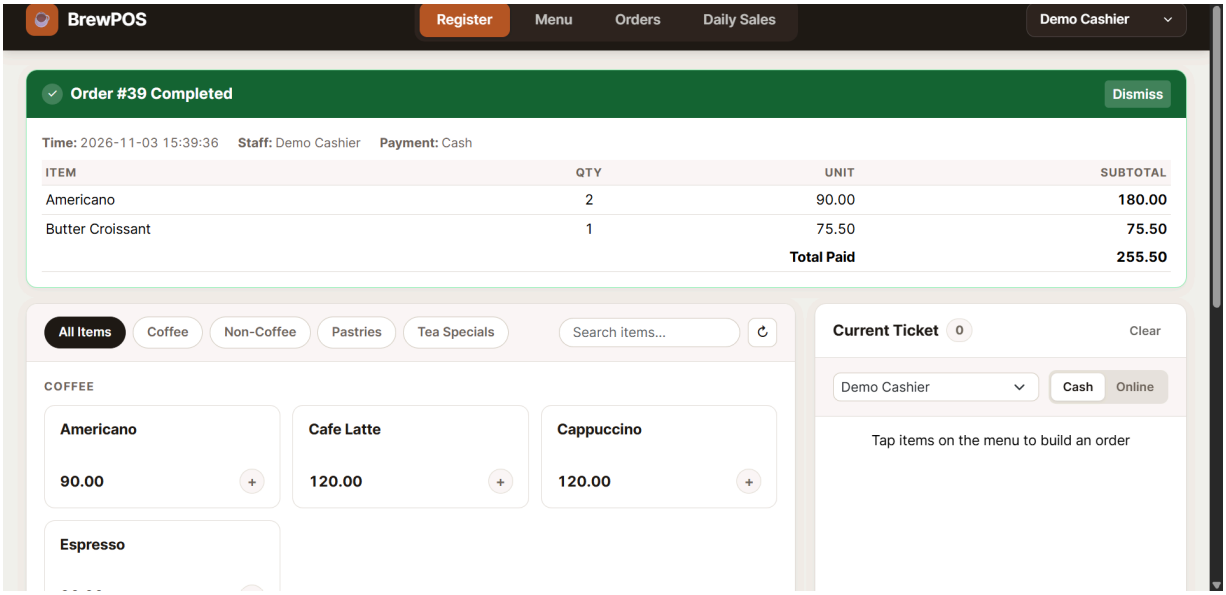
Result / Evidence	Item saved successfully in PostgreSQL with HTTP 201; subsequent PUT returned HTTP 200 with updated price.

Figure 6.4: Cashier Read-Only Restriction Banner



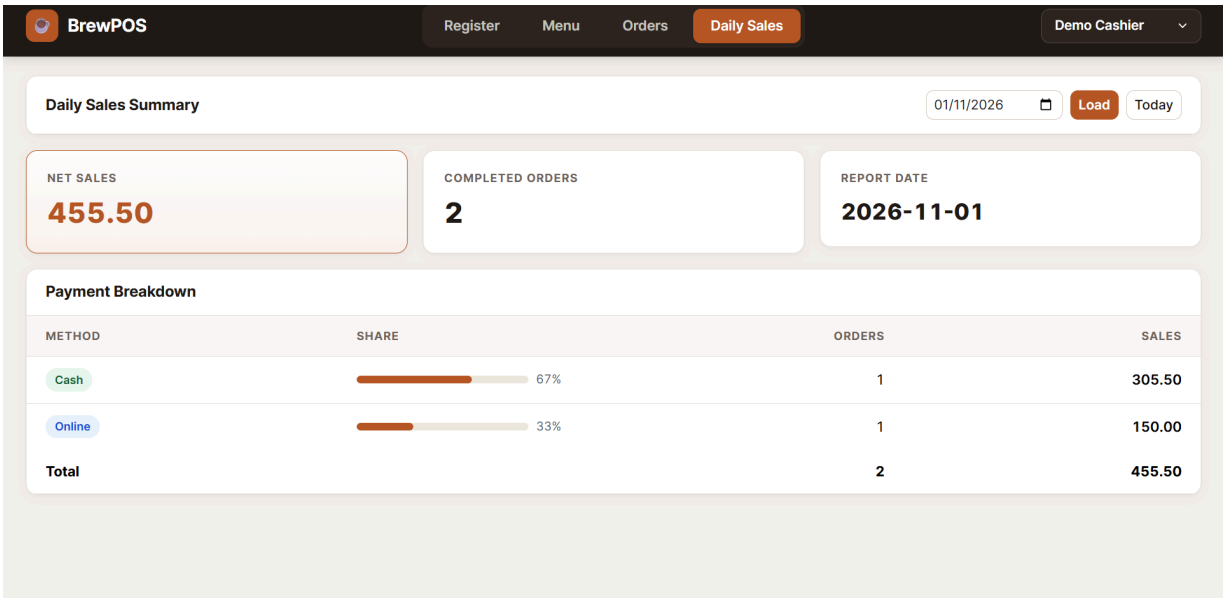
Test Performed	Switched top dropdown to Demo Cashier and navigated to Menu tab.
Result / Evidence	Read-only warning displayed; category and menu item forms and delete buttons were removed from the DOM.

Figure 6.5: Authoritative Itemized Order Receipt



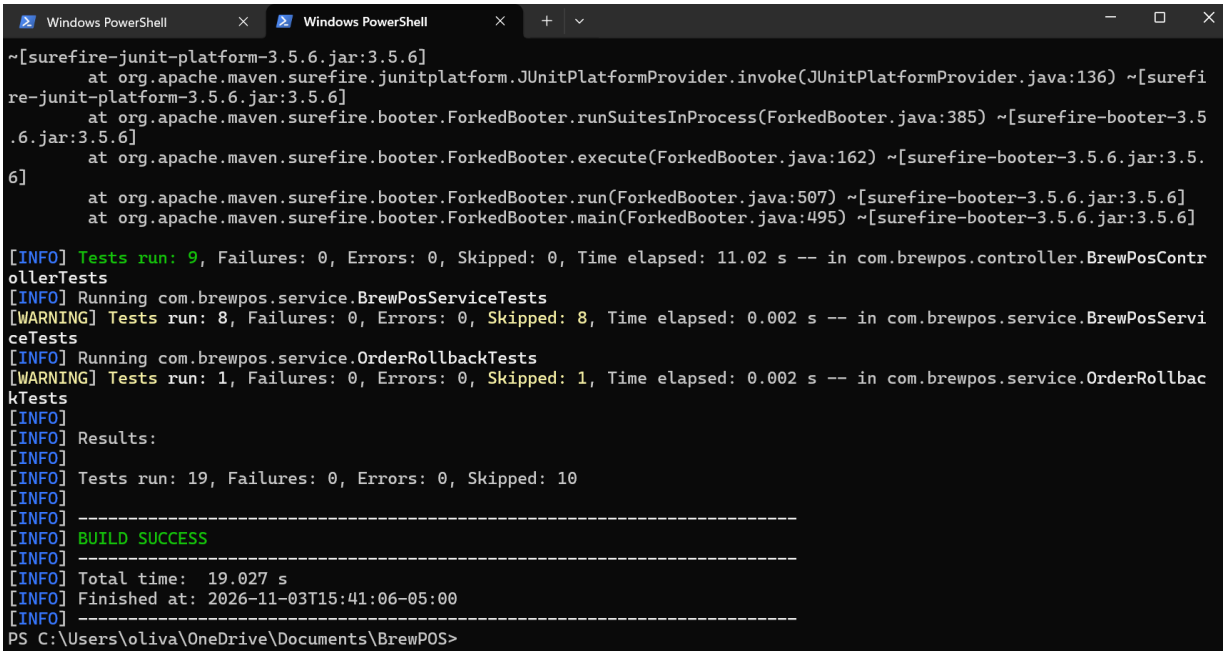
Test Performed	Charged order #1 using Cash tender.
Result / Evidence	PostgreSQL generated transaction ID; receipt modal displayed itemized historical sold prices and dynamically computed total.

Figure 6.6: Daily Sales Report & Tender Breakdown



Test Performed	Loaded daily sales report for current date.
Result / Evidence	Displayed total sales matching SQL query SUM(quantity * sold_price); Cash vs. Online share bar calculated exact percentage shares.

Figure 6.7: Automated JUnit Test Suite Execution



Test Performed	Executed automated suite covering WebMvc controller slices, validation, exception mapping (including HTTP 415), and transactional rollback.
Result / Evidence	Terminal output: Tests run: 19, Failures: 0, Errors: 0, BUILD SUCCESS.

Section 7: Setup and Configuration

7.1 Condensed Installation Steps

1. Clone Repository:

```
git clone https://github.com/Nycri/brewpos
cd BrewPOS
```

2. Initialize Database (grp_brewposdb):

```
psql -U postgres -d postgres -c "CREATE DATABASE grp_brewposdb;"
psql -U postgres -d grp_brewposdb -f database/schema.sql
psql -U postgres -d grp_brewposdb -f database/seed.sql
```

3. Configure Local Environment:

Copy .env.example to .env and set your local PostgreSQL password: Copy-Item .env.example .env (PowerShell) or cp .env.example .env (macOS/Linux).

4. Launch Application:

.\mvnw.cmd spring-boot:run (Windows) or ./mvnw spring-boot:run (macOS/Linux).

5. Access System:

Open browser to http://localhost:8081.

7.2 Environment Variable Contract (Placeholders Only)

Variable	Description	Value
DB_URL	PostgreSQL connection string	jdbc:postgresql://localhost:5432/grp_brewposdb
DB_USERNAME	Database user (postgres or dedicated brewpos_app)	postgres
DB_PASSWORD	Local database password (DO NOT COMMIT REAL PASSWORDS TO GIT)	your_password_here
SERVER_PORT	Web application port	8081

```
# PostgreSQL connection string
DB_URL=jdbc:postgresql://localhost:5432/grp_brewposdb

# Database user (postgres or dedicated brewpos_app)
DB_USERNAME=postgres

# Local database password (DO NOT COMMIT REAL PASSWORDS TO GIT)
DB_PASSWORD=your_password_here

# Web application port
SERVER_PORT=8081
```

7.3 Dedicated Database User and Role Setup

To configure the least-privilege database role in PostgreSQL:

```
psql -U postgres -d grp_brewposdb -f database/05_create_app_role.sql
psql -U postgres -d grp_brewposdb -c "ALTER ROLE brewpos_app WITH PASSWORD
'your_app_password';"
```

Update .env with DB_USERNAME=brewpos_app and DB_PASSWORD=your_app_password.

Section 8: Issues and Resolutions

#	Issue	Problem	Resolution
1	Retrospective Alteration of Past Order Totals on Menu Price Changes	If an item's selling price is modified, joining order_line_item to menu_item.price alters past receipts and misstates financial history.	Added sold_price DECIMAL(10,2) directly to order_line_item. The backend snapshot copies the menu item's active price into sold_price at checkout, permanently insulating historical receipts from catalog updates.
2	Primary Key Violations on Repeated Cart Clicks	Clicking the same product multiple times produced multiple payload lines with identical item_id, violating composite PK (order_id, item_id).	Implemented duplicate line merging in both the client-side cart (pos.js) and server-side order processor (OrderService.java) using a LinkedHashMap with Math::addExact to combine quantities into a single persistent row.
3	IEEE 754 Floating-Point Inaccuracies in Client-Side Money Formatting	JavaScript standard numeric operations produced rounding artifacts (e.g., 0.1 + 0.2 = 0.30000000000000004).	Centralized all monetary arithmetic in integer cents (toCents, formatMoneyFromCents in api.js). In Java, all calculations use BigDecimal with explicit scale 2 and RoundingMode.HALF_UP.
4	Asynchronous Race Conditions in Date Range Filtering	Rapid consecutive clicks on date filters (e.g., clicking "Today" then "All") caused out-of-order fetch resolutions, overwriting the UI with stale records.	Introduced monotonic request sequence counters (listRequestSeq, reportRequestSeq). Incoming HTTP responses check if their sequence matches the latest request; obsolete asynchronous responses are safely discarded.
5	Non-JSON Payloads Triggering Misleading HTTP 500 Responses	Sending requests with wrong Content-Type headers was captured by the generic Exception.class handler, falsely reporting server crashes (HTTP 500).	Added explicit @ExceptionHandler(HttpMediaTypeNotSupportedException.class) returning HTTP 415 (Unsupported Media Type) with sanitized error messaging.

Section 9: AI Usage Declaration

In compliance with academic integrity guidelines, the team declares the following use of generative AI development tools during the completion of Activity 3:

9.1 AI Tools Employed

1. Antigravity / Gemini 3.8 Flash & Argon
2. Anthropic Claude 3.5 Sonnet and ChatGPT (Architecture Review)

9.2 Tasks Performed with AI Assistance

1. Scaffolding baseline Spring Boot DTO records and REST controller signatures.
2. Formulating custom CSS color harmonies and high-contrast POS button classes.
3. Drafting boilerplate JUnit slice assertions for HTTP response validation.
4. Technical proofreading and structured documentation compilation.

9.3 Validation, Verification & Correction Procedures

Procedure	Description
Constraint & Schema Verification	All AI-suggested SQL constructs were manually verified against the strict 5-table boundary and PostgreSQL 18 syntax. Proposals suggesting additional tables or surrogate keys were rejected.
Codebase Security Review	AI-generated code was manually audited to ensure that no client-submitted prices or totals were accepted by the backend, and all DOM elements were constructed using textContent to prevent Cross-Site Scripting (XSS).
Empirical Test Suite Execution	All code refactors were verified by executing the Maven automated test suite (.\mvnw.cmd clean test), confirming 100% test pass rates prior to project finalization and submission.